

Entendendo Algoritmos

Nicolas Ramos Carreira

Sumário

0	Sobre o livro	2
0.1	Panorama geral dos capitulos	2
0.2	Como usar o livro	3
0.3	Quem deve ler o livro	3
1	Introdução a algoritmos	4
1.1	Introdução	4
1.1.1	O que aprenderemos sobre desempenho	4
1.1.2	O que aprenderemos sobre a solução de problemas	4
1.2	Pesquisa binária	4
1.2.1	Uma maneira melhor de buscar (a pesquisa binária)	5
1.2.2	Tempo de execução	9
1.3	Notação Big O	9
1.3.1	Tempo de execução dos algoritmos cresce a taxas diferentes	9
1.3.2	Vendo diferentes tempos de execução Big O	10
1.3.3	A notação Big O estabelece o tempo de execução para a pior hipótese	11
1.3.4	Alguns exemplos comuns de execução Big O	11
1.3.5	O caixeiro-viajante	12
1.4	Resumo do capítulo	13
2	Ordenação por seleção	14
2.1	Como funciona a memória	14
2.2	Arrays e listas encadeadas	15
2.2.1	Listas encadeadas	16
2.2.2	Arrays	17
2.2.3	Terminologia e alguns detalhes	17
2.2.4	Inserindo algo no meio da lista	18
2.2.5	Deleções	19
2.3	Ordenação por seleção	19
2.4	Resumindo	19

Capítulo 0

Sobre o livro

O livro, em seu início, já destaca qual a sua ideia central, que é: Ser um livro de fácil leitura, que irá pegar conteúdos complexos e simplificar através das explicações, exemplos, ilustrações e prática ao longo do livro. O livro deixa claro que não aborda todos os algoritmos existentes, mas sim os mais importantes e considerado úteis pelo autor.

0.1 Panorama geral dos capitulos

Os primeiros 3 capítulos do livro se constituirão no seguinte:

- Capítulo 1: Aprenderemos o nosso primeiro algoritmo básico, a busca binária. Além disso, veremos como analisar a velocidade de um algoritmo utilizando a notação Big-O (que será utilizada ao longo do livro inteiro)
- Capítulo 2: Aprenderemos duas estruturas de dados fundamentais, que são os arrays e listas encadeadas. Elas também são usadas na criação de estruturas de dados mais avançadas, como a tabela hash
- Capítulo 3: Aprenderemos o uso da recursão, uma técnica muito útil utilizada em muitos algoritmos

Os capitulos acima são os mais importantes (principalmente por conta da notação Big-O e da recursão), portanto, são os que o autor vai em ritmo mais lento.

O restante do livro apresenta alguns algoritmos de aplicação mais ampla. Veja:

- Tecnicas para resolução de problemas: Abordadas nos capítulos 4, 8, 9. São abordadas técnicas, como divisão e conquista (cap 4), programação dinamica (cap 9) e algoritmo guloso (cap 8) para problemas que não sabemos bem como resolvê-lo de forma eficiente.

- Tabela Hash: É uma estrutura de dados muito útil que é abordada no capítulo 5
- Algoritmos de grafos: Abordados nos capítulos 6 e 7, grafos são uma maneira de modelar uma rede. Veremos sobre a pesquisa em largura (cap 6) e algoritmo de Dijkstra (cap 7)
- K-vizinhos mais próximos: Abordado no capítulo 7, essa é uma técnica simples de aprendizado de máquina. Podemos utiliza-la para criar recomendações de sistema, mecanismo OCR e até um sistema para prever valores (ou seja, tudo que envolve prever um valor).
- Proximos passos: O capítulo 11 é percorrido sobre dez algoritmos que valem a pena uma leitura posterior (quando você já estiver craque em algoritmos)

0.2 Como usar o livro

O autor se preocupou bastante com a ordem com que os assuntos seriam abordados, sendo assim, o ideal é que se leia os capítulos em ordem (eles se baseiam uns nos outros).

Execute o código dos exemplos. Isso te fará reter melhor os conteúdos abordados. Você pode baixa-los no github através [DESTE LINK](#) (nesse repositório, o autor disponibilizou os códigos em várias linguagens, como C#, Python, Ruby..). Uma observação é que os exemplos abordados utilizam Python como linguagem.

Obviamente que é primordial que os exercícios passados sejam feitos. Eles nos ajudarão a conferir nosso pensamento (se estamos seguindo a linha de raciocínio correta ou não)

OBS: Um detalhe é que EU, Nicolas, gostaria de deixar registrado a forma de estudo que eu usei para estudar o livro. Além de fazer o que foi falado acima, para os conteúdos teóricos, irei ler, entender e depois passar por escrito aqui para o LATEX

0.3 Quem deve ler o livro

É destinado a qualquer um que queira aprender sobre programação e imergir no mundo dos algoritmos

Capítulo 1

Introdução a algoritmos

Neste capítulo, aprenderemos:

- Como fazer uma busca binária
- Entender o uso da notação Big-O para analisar a velocidade de algoritmos

1.1 Introdução

Um algoritmo é o conjunto de instruções para realizar determinada tarefa. Cada trecho de um código poderia ser um algoritmo, mas este livro se concentra nos mais importantes. Os algoritmos apresentados no livro foram escolhidos porque são rápidos ou porque resolver problemas interessantes.

Em cada um dos casos, o autor irá descrever o algoritmo e apresentar um exemplo. Em seguida, será falado sobre o tempo de execução do algoritmo em notação Big-O. Por fim, serão explorados outros casos de uso (problemas) onde há aplicabilidade daquele algoritmo

1.1.1 O que aprenderemos sobre desempenho

Aprenderemos, principalmente, a comparar o desempenho de diferentes algoritmos. Exemplo: "Para este caso, devemos usar quicksort ou mergesort. Devemos usar uma lista encadeada ou um array?"

1.1.2 O que aprenderemos sobre a solução de problemas

1.2 Pesquisa binária

Antes de tudo, vamos a um exemplo: Suponhamos que tenhamos uma lista telefonica e queremos procurar um contato com a letra K, poderíamos começar folheando a lista até encontrar ou podemos começar do meio (já que sabemos que

não estará no começo) e partir dali.

Outro exemplo interessante é o Facebook. Suponhamos que o Facebook quer verificar se determinada conta existe. O nome da conta é Nicolas. Ele irá procurar no banco de dados. Poderia até começar do A, mas faz mais sentido começar a busca pelo meio.

Todos os exemplos acima representam a aplicabilidade da busca binária. A busca binária é um algoritmo, onde passamos para ele uma lista de elementos (que deverá estar ordenada) e se o elemento buscado está na lista, será retornada a posição do elemento e caso contrário, será retornado None

Um exemplo interessante para visualizarmos é: suponhamos que tivéssemos uma lista onde temos números de 1 a 100 e eu peço a alguém que tente acertar o número que estou pensando (nesse caso 99), onde digo, a cada tentativa, se o número chutado está muito baixo ou muito alto. Uma maneira que a pessoa possa pensar para encontrar o número que estou pensando é chutar número a número começando do 1. Esse método se chama pesquisa simples (ou pesquisa estúpida como disse o autor). Essa é uma maneira pouco eficiente, sendo 99 o número que escolhi, a pessoa precisaria chutar 99 números para acertar. A cada chute, ela estaria eliminando apenas um número por vez dessa maneira.

1.2.1 Uma maneira melhor de buscar (a pesquisa binária)

Dessa forma, uma maneira mais eficiente de acertar o número que estou pensando seria começar chutando do 50, assim, eu diria "muito baixo". Com isso, a pessoa eliminaria METADE dos números, pois ela saberia que os números de 1 a 50 são muito baixos. Aí depois, suponhamos que ela chute 75 e (como meu número é 99) eu fale "muito baixo". Ainda sim, ela eliminou uma quantidade considerável de valores. Isso continuaria, até que enfim meu número fosse encontrado. Essa maneira de pesquisar se chama pesquisa binária.

Você percebe portanto que na pesquisa binária, a busca ocorre através dos valores intermediários, o que permite eliminar uma grande quantidade de valores a cada etapa

Suponha agora que você esteja procurando uma palavra em um dicionário com 240.000 palavras. Na pior das hipóteses (a última palavra desse dicionário), em cada uma das formas de busca:

- Pesquisa simples: 240.000 etapas
- Pesquisa binária: 18 etapas, uma vez que a cada etapa eliminamos o número de palavras pela metade, até que sobre apenas uma palavra.

Portanto, percebe-se uma grande diferença! Podemos dizer que a pesquisa binária, de maneira geral, para uma lista com n elementos, ela precisa de $\log_2 n$

para retornar o valor correto, enquanto isso, a pesquisa simples precisaria de n etapas. Exemplo: em uma lista com 8 elementos, na pesquisa simples, precisaríamos checar, em seu máximo, 8 elementos. Enquanto isso, na pesquisa binária, precisaríamos, em seu máximo, checar $\log_2 8$, que seriam 3 elementos

OBS: na notação Big O sempre quando falamos de log, na verdade estamos nos referindo a $\log_2$

Um detalhe que já chegamos a comentar é que para que a pesquisa binária ocorra, a nossa lista precisa estar ordenada.

Implementação da pesquisa binária

Agora, para conseguir visualizar melhor a pesquisa binária, realizaremos sua implementação. Faremos a implementação em um array. Teremos a função `pesquisa_binaria`, que receberá um array ordenado e um valor. Se o valor estiver no array, será retornada sua posição. Caso contrário, será retornado `None`.

No começo da nossa função `pesquisa_binaria`, teremos:

```
baixo = 0
alto = len(lista) - 1
```

Acima, acontece o mapeamento das posições do array, para assim conseguirmos pegar a posição intermediária. Após isso, teremos:

```
meio = (baixo + alto) // 2
chute = lista[meio]
```

O que ele faz acima é encontrar o meio do nosso array e depois pegar o valor que temos no meio dele. Perceba que utilizamos o operador `//`, ou seja, estamos fazendo uma divisão inteira. Isso ocorre para que o valor "meio" seja arredondado para baixo caso $(baixo + alto)$ não seja um valor par. A partir disso, temos:

```
if chute < item:
    baixo = meio + 1
```

Se o valor do primeiro chute for MENOR que o item que estamos procurando, então o valor "baixo" é atualizado para $meio + 1$. Isso acontece porque ele passa a desconsiderar todos os valores do antigo meio para baixo. O mesmo ocorre para caso o valor do primeiro chute for MAIOR que o item que estamos procurando:

```
if chute > item:
    alto = meio - 1
```

O que acontece acima é que se o valor do chute for maior que o item que estamos procurando, o valor "alto" é atualizado, pois ele passa a desconsiderar todos os valores do antigo meio para cima

Agora veja o código completo abaixo:

```
def pesquisa_binaria(lista, item):
    baixo = 0
    alto = len(lista) - 1

    while baixo <= alto:
        meio = (baixo + alto)//2
        chute = lista[meio]

        if chute == item:
            return meio

        if chute > item:
            alto = meio - 1

        else:
            baixo = meio + 1

    return None
```

Agora, suponhamos que tenhamos a lista [1, 3, 5, 7, 9], onde tentaremos buscar o valor 3. Com base no código, acima, o que aconteceria seria o seguinte:

1. Em

```
    baixo = 0
    alto = len(lista) - 1
```

a variável "baixo" iria ser definida como 0 e a variável "alto" seria definida como 4 (índice final do array). Essas duas variáveis acompanham a parte da lista que estamos procurando

2. Após isso, temos:

```
    while baixo <= alto:
        meio = (baixo + alto) // 2
        chute = lista[meio]
```

onde enquanto ele não terminar a busca em toda a lista, ele irá verificar o elemento central. No nosso caso, o valor de meio inicialmente será 2 (resultado da divisão por 2 de $0 + 4$) e portanto, nosso chute será 5.

3. Como o chute verificado não é igual a 3 (nosso item) e é maior que o valor do item, ele partirá a segunda condição onde:

```
if chute > item:
    alto = meio - 1
```

com isso, todos os valores de 5 em diante da nossa lista (5, 7, 9) serão desconsiderados, pois o valor da nossa variável alto passará a ser 1 (ou seja, o índice 1 da nossa lista, que tem 3 como valor).

4. Depois disso, voltaremos ao nosso while (uma vez que temos a nossa variável "baixo" como 0 e nossa variável "alto" como 1)

```
while baixo <= alto:
    meio = (baixo + alto) // 2
    chute = lista[meio]
```

A partir disso, nosso meio será igual a 0, pois $(0 + 1)/2$ é igual a 0.5, mas como é arredondado para baixo, será igual a 0. Sendo assim, nosso chute será igual a 1.

5. Como nosso chute não é igual ao valor do item (3) e é menor que o valor do item, ele partirá para a terceira condição, onde:

```
else:
    baixo = meio + 1
```

com isso, o valor da nossa variável baixo passará a ser 1, pois o meio era 0.

6. Com isso, voltaremos ao while pois a condição ainda é atendida (variável "baixo" é 1 e variável "alto" é 1, então $1 \leq 1$)

```
while baixo <= alto:
    meio = (baixo + alto) // 2
    chute = lista[meio]
```

nosso meio agora passará a ser 1 (pois $(1 + 1) // 2$ é igual a 1) e então nosso chute será o valor 3

7. Agora, como nosso chute é igual ao item (3), ele entra na primeira condição e retorna o valor a variável meio (que é, no caso, o índice do valor do chute), encerrando a função e a busca

```
if chute == item:
    return meio
```

Caso queira ver e testar por si só, basta acessar o código do algoritmo [NESTE LINK](#)

1.2.2 Tempo de execução

Sempre quando falamos sobre um algoritmo, falamos sobre o seu tempo de execução. Geralmente, escolhemos o algoritmo mais eficiente, a fim de otimizar tempo e espaço. Vamos pegar como exemplos as pesquisas simples e binária que falamos anteriormente.

Na pesquisa simples, o número máximo de tentativas (etapas) é igual ao tamanho da lista, o que caracteriza o tempo de execução **LINEAR**. Se temos uma lista com 100 elementos, precisaremos, em seu pior caso, de 100 tentativas.

Nas pesquisa binária, se temos uma lista com 100 elementos, precisaremos de no máximo 7 tentativas. A pesquisa binária é executada com tempo **LOGARÍTMICO**.

1.3 Notação Big O

A notação Big O é uma notação especial que diz o quão rápido é um algoritmo. A importância dele é que frequentemente estaremos utilizando algoritmos de outras pessoas e quando fazemos isso, é importante entender o quão rápido ou lento esse algoritmo é.

1.3.1 Tempo de execução dos algoritmos cresce a taxas diferentes

Suponhamos que temos um problema onde podemos implementar o algoritmo de pesquisa simples e pesquisa binária para uma lista com 100 elementos.

Implementando a pesquisa simples para a lista de 100 elementos, suponhamos que levamos 100ms. Enquanto isso, a pesquisa binária, para a mesma lista de 100 elementos, poderia levar 7ms aproximadamente.

Você pode pensar: puxa, a pesquisa binária é cerca de 15 vezes mais rápida que a pesquisa simples, então para um problema onde tenho uma lista de 1 bilhão de elementos, se a pesquisa binária demorar 30ms, então a pesquisa simples demoraria $15 * 30\text{ms} = 450\text{ms}$, correto?

Não. Na verdade, está completamente errado. Isso acontece porque o tempo de execução da pesquisa simples e binária crescem com taxas diferentes. Veja a imagem abaixo:

	PESQUISA SIMPLES	PESQUISA BINÁRIA
100 ELEMENTOS	100 ms	7 ms
10000 ELEMENTOS	10 segundos	14 ms
1,000,000,000 ELEMENTOS	11 dias	32 ms

Tempos de execução crescem com velocidades diferentes!

Portanto, perceba que conforme o número de elementos na lista cresce, a pesquisa binária aumenta só um pouco o seu tempo de execução. Por outro lado, a pesquisa simples irá levar muito tempo a mais.

Dessa forma, não basta saber quanto tempo um algoritmo leva para ser executado, é necessário saber se o tempo de execução aumenta conforme a lista aumenta. É aí que entra a notação Big O.

A notação Big O não utiliza segundos para medir quão rápido é um algoritmo, ela informa o número de operações, permitindo que você veja o quão rapidamente um algoritmo cresce. Nas pesquisa simples, por exemplo, para uma lista com n elementos, teremos um Big $O(n)$, assim, se temos 100 elementos, temos $O(100)$, se temos 10000 elementos, temos um $O(10000)$. Isso é muito menos propenso a erros do que o modo anterior onde olhávamos os segundos (e eu mostrei que poderia haver confusões)

Sendo assim a notação Big O nos mostra o número de operações que um algoritmo realiza. É chamado de Big O porque coloca-se um grande O na frente do número de operações

1.3.2 Vendo diferentes tempos de execução Big O

Falaremos agora de um exemplo prático abordado pelo livro para visualizar diferentes tipos de execução Big-O.

O exemplo é basicamente pegar uma folha de papel e formar nela 16 quadrados. Poderíamos fazer isso de duas formas:

1. Pegar um lápis e fazer quadrado a quadrado.
2. Fazer dobras com o papel para que quando desdobrassemos, estivessem os 16 quadrados nele formados pelas dobras

Olhando as duas formas de fazer isso e considerando que o Big-O mede o número de operações, podemos ver que a primeira forma resultaria em um número grande de operações, mais precisamente 16 operações. Enquanto isso, a

segunda forma levaria 4 dobras, ou seja, 4 operações

Dessa forma, podemos dizer que da primeira maneira o Big-O desse "algoritmo" é $O(n)$, pois teríamos que fazer quadrado a quadrado.

Por outro lado, fazendo da segunda maneira, podemos dizer que temos um "algoritmo" de $O(\log_2 n)$, uma vez que para 16 quadrados, teríamos 4 operações.

1.3.3 A notação Big O estabelece o tempo de execução para a pior hipótese

Suponhamos que nós estamos utilizando a pesquisa simples para procurar um nome em uma lista telefônica. Sabemos que a pesquisa simples tem um tempo de execução de $O(n)$, ou seja, no pior caso, teríamos que percorrer a lista inteira. Agora suponha que o contato que estamos buscando tem o nome "Ana" e é o primeiro contato da lista. Nós teríamos um tempo de execução de $O(n)$ ou $O(1)$?

Justamente para que não haja essa ambiguidade, a notação Big O mede o tempo de execução a partir da pior hipótese. Olhar para o pior caso é uma garantia, você sabe que não terá tempo de execução mais lento que $O(n)$ para a pesquisa simples, por exemplo

OBS: analisar o caso médio também é interessante. Será discutido no livro mais afrente

1.3.4 Alguns exemplos comuns de execução Big O

Abaixo, teremos cinco exemplos de execução Big O que encontraremos ao longo do livro e até ao longo de nossa vida. Eles estão ordenados do mais rápido para o mais lento:

- $O(\log_2 n)$: Também conhecido como tempo logarítmico. Exemplo: pesquisa binária
- $O(n)$: conhecido como tempo linear. Exemplo: pesquisa simples
- $O(n * \log_2 n)$: Um exemplo é o algoritmo rápido de ordenação quicksort que usa essa notação
- $O(n^2)$: Podemos citar como exemplo o algoritmo lento de ordenação, como a ordenação por seleção
- $O(n!)$: Temos o exemplo do algoritmo bem lento caixeiro-viajante

Além desses tempo de execução Big-O, existem outros, mas os que foram abordados são os mais comuns.

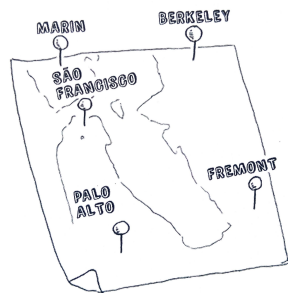
Sendo assim, os principais pontos sobre Big-O são:

- A rapidez de um algoritmo não é medida em segundos, mas sim pelo crescimento do número de operações
- Um $O(\log_2 n)$ é mais rápido que o $O(n)$ e fica ainda mais rápido conforme a lista aumenta

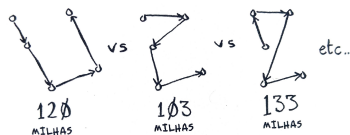
1.3.5 O caixeiro-viajante

Aqui temos um exemplo de algoritmo com o tempo de execução muito ruim. Estamos tratando de um problema clássico da ciência da computação, pois seu crescimento é apavorante e muitos estudiosos acreditam que seja impossível de melhorá-lo. Vamos a ele:

Nós temos um caixeiro-viajante que precisa ir a 5 cidades suponhamos.



Ele precisa passar por todas as cidades percorrendo a distância mínima. Com isso, existem algumas ordens de cidades para visitar. Para 5 cidades, existem 120 permutações possíveis (ou seja, 120 ordens de cidades possíveis)



Logo, precisamos de 120 operações para resolver o problema das cinco cidades (calcular e comparar). Para seis cidades, precisaríamos de 720 operações (ou 720 permutações), para sete 5050 e por aí vai.

CIDADES	OPERAÇÕES
6	720
7	5040
8	40320
...	...
15	1307674368000
...	...
30	265252859812191058636308880000000

O número de operações aumenta drasticamente.

Com isso, para n itens, precisaríamos de $n!$ operações para chegar ao resultado, sendo então o tempo de execução um $O(n!)$, assim como tínhamos visto. O que acontece é que a partir de um número de cidades (100 cidades, por exemplo), é IMPOSSIVEL calcular a resposta em função do tempo (são MUITAS operações)

Aí fica a pergunta: mas não tem como usar outro algoritmo para isso? Não, pois esse problema não tem solução, não existe um algoritmo mais rápido para esse problema e estudiosos acreditam que não existe maneiras de melhorá-lo. O que se pode fazer é encontrar uma solução aproximada que veremos no capítulo 10 do livro.

1.4 Resumo do capítulo

Ao longo desse capítulo vimos muitas coisas, dentre elas:

- Abordamos rapidamente o conceito de algoritmos
- Vimos um pouco sobre a pesquisa simples e binária
- Aprofundamos na pesquisa binária vendo inclusive sua implementação
- Vimos sobre tempo de execução de algoritmos e o porquê que não o medimos em segundos, mas sim a partir da notação Big-O
- Aprofundamos na notação Big-O, destacando que ela é uma notação que mostra o número de etapas necessárias
- Abordamos os diferentes Big-O (os mais usados)
- Abordamos o problema do caixeiro-viajante, onde temos um algoritmo $n!$ que não pode ser melhorado e não há muito o que fazer com relação a isso

Capítulo 2

Ordenação por seleção

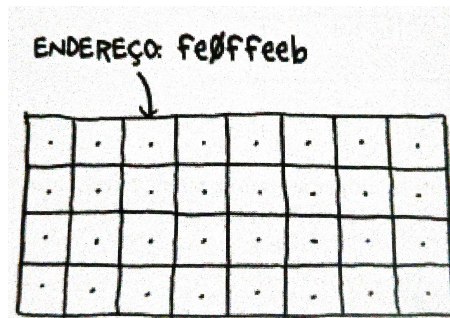
Neste capítulo aprenderemos:

- Veremos sobre as duas estruturas de dados essenciais: arrays e listas encadeadas. No primeiro capítulo chegamos a abordar rapidamente os arrays para falar sobre a pesquisa simples e binária, porém esse capítulo trará uma profundidade maior, abordando os prós e contras de cada uma, por exemplo.
- Veremos também nosso primeiro algoritmo de ordenação, a ordenação por seleção. É importante isso porque vários algoritmos só funcionam se os dados estiverem ordenados. Além disso, o que vemos aqui servirá de base para o próximo capítulo, onde aprenderemos o quicksort

2.1 Como funciona a memória

Suponhamos que você tem um armário com várias gavetas e precisa guardar algumas coisas nele. Em cada gaveta, você poderá guardar um elemento, então suponha que você tem uma gravata e um chapéu, você irá guardar o chapéu em uma gaveta e a gravata em outra

A memória de um computador funciona de forma parecida com o exemplo acima. A memória é um armário com várias gavetas, sendo que cada gaveta tem seu endereço



Cada vez que você precisa armazenar um item na memória, você pede ao computador espaço e ele te dá um endereço onde você pode armazenar aquele item.

2.2 Arrays e listas encadeadas

As vezes pode acontecer de querermos armazenar múltiplos itens na memória. Suponha que você esteja fazendo um app de tarefas, você precisará armazenar as tarefas como se fosse uma lista na memória. Podemos fazer de duas formas: usando arrays e listas encadeadas.

Usando um array, significa que todas as nossas tarefas ficam armazenadas contiguamente, ou seja, cada tarefa é armazenada uma ao lado da outra na memória, assim como podemos ver na imagem abaixo.



O problema disso é que, olhando a imagem acima, por exemplo, se depois de "Chá", quisermos armazenar outra tarefa na memória, não conseguiremos, pois já está ocupada. É como se você fosse ao cinema com seus amigos e sentassem um ao lado do outro e um outro amigo chegasse de última hora, mas não conseguisse sentar ao lado de vocês por estar ocupado.

Seria então necessário solicitar ao computador uma área de memória em que coubessem todas as tarefas contiguamente. A questão é que essa situação poderia acontecer novamente, o que pode tornar a adição de itens no array um

processo muito lento.

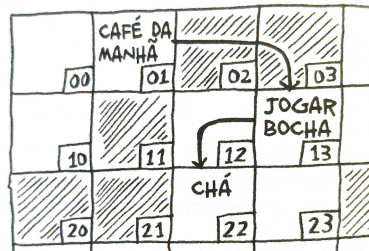
Com isso, uma maneira relativamente simples de resolver isso é: "reservar lugares". Mesmo que você tenha três itens na lista de tarefas, você pode solicitar ao computador dez espaços, só por via das dúvidas. Dessa forma, você consegue adicionar 10 elementos a sua lista sem precisar mover nada, mas ainda existem desvantagens:

- Você pode não precisar dos espaços extras que reservou, então a memória seria desperdiçada
- Você pode ter que utilizar mais de dez espaços, então aquela situação voltaria a acontecer

Dessa forma, apesar de ser uma maneira de contornar o problema, não é uma solução perfeita. O ideal seria resolver isso com listas encadeadas

2.2.1 Listas encadeadas

Nas listas encadeadas, seus itens podem estar em qualquer lugar da memória. Como cada item armazena o endereço do próximo item da lista, eles não precisam estar contíguos, eles ficam espalhados. Veja a imagem abaixo:



Endereços de memória ligados.

Uma analogia que pode ser feita é com uma caça aos tesouros. Você vai primeiro em endereço x ele diz que o próximo item pode ser encontrado no endereço y

Adicionar elementos em uma lista encadeada é algo bem simples. Você basicamente coloca em qualquer lugar da memória e armazena o endereço do item anterior.

Uma coisa interessante é que com as listas encadeadas você não tem o problema de ter que mover os itens. Além disso, evitamos outro problema onde suponhamos que estamos tentando encontrar 10.000 slots da memória pra um array. Se sua memória tem os 10.000 slots, mas eles não estão juntos, você não

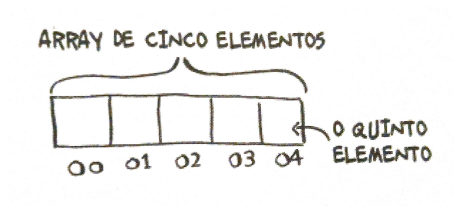
consegue arrumar um lugar para o seu array, o que definitivamente não seria um problema, pois ele espalharia os 10.000 itens pela memória

Agora, fica uma questão, se as listas encadeadas são melhores em inserções, para que usar arrays?

A questão é que as listas encadeadas ainda tem limitações. Suponhamos que você queira ler o último item de uma lista encadeada, você deverá passar pelo primeiro item, para pegar o endereço para o item dois e ir para ele e assim consecutivamente, até ir para o último. Quando falamos de arrays, já não temos essa limitação

2.2.2 Arrays

Nos arrays, você sabe o endereço de cada item, então você não tem a limitação que as listas encadeadas tem. Se você tem um array com cinco itens, sabe que o primeiro está no endereço 00 e quer acessar o último elemento, qual seria o endereço? Seria o endereço 04. Dessa forma, arrays são ótimos se você deseja encontrar elementos aleatórios, pois pode encontrar qualquer elemento instantaneamente com array



2.2.3 Terminologia e alguns detalhes

Algo que pode ter passado despercebido é a a numeração do array começando do 0. Isso é uma convenção que é utilizada em todas as linguagens de programação. Além disso, um detalhe importante de se pontuar é que ao invés de dizer "o número x está na posição 1 do array", seguiremos a terminologia correta e diremos: "o número x está no ÍNDICE 1 do array". Logo, posição = índice. O livro usará essa terminologia o tempo todo a partir de agora

Outro detalhe importante de se citar é o comparativo em relação ao tempo de execução das operações básicas (leitura e inserção) de arrays e listas. Esse tópico será abordado com mais profundidade em breve (ainda neste capítulo), porém é importante ter uma ideia disso, pois é utilizado para realizar o exercício 2.1 do livro. Desta forma, fique com a imagem abaixo:

	ARRAYS	LISTAS
LEITURA	$O(1)$	$O(n)$
INSERÇÃO	$O(n)$	$O(1)$

$O(n)$ = TEMPO DE EXECUÇÃO LINEAR
 $O(1)$ = TEMPO DE EXECUÇÃO CONSTANTE

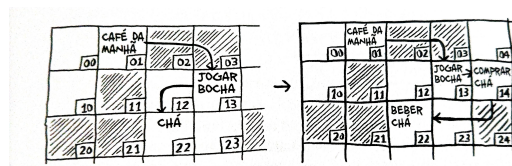
2.2.4 Inserindo algo no meio da lista

Suponha que você tem uma lista de tarefas assim como abordamos nos exemplos anteriores, mas agora você quer colocar as tarefas na ordem em que elas serão feitas

Se você quiser inserir um elemento no meio da sua lista, o que seria melhor arrays ou listas encadeadas?

Em listas encadeadas

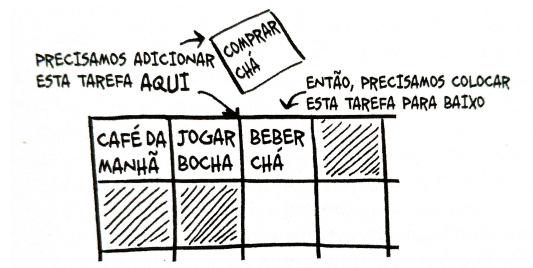
Em listas encadeadas, bastaria mudar o endereço para qual o elemento anterior está apontando, para que ele passe a apontar para o novo elemento que você adicionou. Veja a imagem de exemplo abaixo:



Note que antes, "Beber chá" vinha logo depois de "Jogar bocha" (ou seja, jogar bocha apontava para beber chá), mas com a adição de "Comprar chá", houve uma modificação e "Jogar Bocha" passou a apontar para "Comprar chá"

Em arrays

Nos arrays, ao inserir um elemento no meio dele, precisamos mover todos os itens que estão a partir do endereço de inserção. Veja a imagem abaixo:



Suponhamos que o elemento "Café da manhã" seja índice 0, "Jogar bocha" índice 1 e "Beber chá" índice 2. A partir disso, conforme a imagem mostra, vamos dizer que queremos inserir "Comprar chá" no índice 2, no lugar de "Beber chá". O que aconteceria é que "Beber chá" teria que ser movido para a dar lugar a "Comprar chá"

O problema disso tudo é que se não houver espaço, pode ser necessário mover tudo para um novo local na memória. Por isso, listas encadeadas acabam levando vantagem caso você queira inserir um elemento no meio de uma lista

2.2.5 Deleções

2.3 Ordenação por seleção

2.4 Recapitulando